



Proyecto Unidad 2 y 3 - Internet de las Cosas SmartHome IoT - Protocolos avanzados, seguridad e inteligencia local con LLM

Ricardo Pérez
riperez@utalca.cl

Unidad 2: Protocolos de Comunicación, Seguridad y LLM Local

1. Introducción

En esta segunda fase se extiende el sistema SmartHome construido en la Unidad 1 con dos mejoras fundamentales. Primero, se reemplaza la comunicación MQTT básica por una versión con autenticación y cifrado TLS, y se experimenta con CoAP como protocolo alternativo orientado a dispositivos con recursos limitados. Segundo, se incorpora un **Modelo de Lenguaje Local (LLM)** ejecutándose completamente en el computador del equipo, sin acceso a internet, que actuará como el “cerebro” del sistema SmartHome: interpretará el estado del hogar, tomará decisiones de control y responderá preguntas en lenguaje natural.

2. Problema a resolver

El sistema SmartHome de la Unidad 1 funciona, pero sus reglas de control son rígidas (estructuras *if/else* fijas). La familia ahora quiere:

- Poder **consultar el sistema** en lenguaje natural: “¿Cuál es la temperatura del living?”, “¿Hay alguien en la casa?”.
- Que el sistema tome **decisiones contextuales e inteligentes**: si es de noche, hay movimiento y el gas está elevado, el LLM decide qué actuadores activar y redacta el mensaje de alerta.
- Que la comunicación sea **segura**: MQTT con autenticación y cifrado extremo a extremo.
- Explorar un protocolo alternativo (**CoAP**) para al menos un sensor de bajo consumo.

3. Objetivo

Extender el sistema SmartHome con protocolos de comunicación seguros (MQTT-TLS, CoAP), integrar un modelo de lenguaje ejecutándose localmente (Ollama) para automatización intelligen-

te y consultas en lenguaje natural, y asegurar la infraestructura IoT mediante buenas prácticas de seguridad.

4. Objetivos específicos

1. Configurar el broker Mosquitto con autenticación por usuario/contraseña y cifrado TLS.
2. Actualizar el firmware del ESP32 y los flujos de Node-RED para conectarse al broker seguro.
3. Implementar CoAP en al menos un nodo sensor ESP32 y comparar su rendimiento con MQTT.
4. Instalar y configurar Ollama con un modelo liviano (`phi3:mini` o `llama3.2:3b`) para inferencia local.
5. Integrar el LLM en Node-RED para que reemplace las reglas *if/else* por razonamiento en lenguaje natural.
6. Agregar al *dashboard* un campo de consulta en lenguaje natural respondido por el LLM.
7. Documentar las medidas de seguridad implementadas y analizar al menos una vulnerabilidad del stack IoT.
8. Extender el informe técnico con las secciones de protocolos, seguridad y LLM.

5. Funcionalidades mínimas requeridas

5.1 MQTT seguro con autenticación y TLS

1. Configurar Mosquitto con usuario y contraseña usando `mosquitto_passwd`.
2. Generar un certificado TLS autofirmado y activar el broker en el puerto **8883**.
3. Actualizar el firmware del ESP32 y los flujos de Node-RED para conectarse con TLS.
4. Configurar ACLs en Mosquitto para limitar qué clientes pueden publicar o suscribirse a cada tópico.

Los mensajes mantienen el mismo formato JSON y la misma jerarquía de tópicos de la Unidad 1. Se agregan dos tópicos nuevos:

```
smarthome/equipoXX/llm/decision
smarthome/equipoXX/llm/respuesta
```

Ejemplo de generación del certificado autofirmado:

```
openssl req -new -x509 -days 365 -extensions v3_ca \
-keyout ca.key -out ca.crt
```

5.2 CoAP en al menos un nodo sensor

- Configurar un ESP32 para publicar datos de temperatura o humedad usando el protocolo **CoAP** (puerto UDP 5683) mediante la librería **CoAP-simple-library**.
- En Node-RED, recibir los recursos CoAP con el nodo **node-red-contrib-coap** y redirigir los datos al broker MQTT.
- Registrar en el informe una comparación del tamaño de los mensajes y el consumo de ancho de banda entre MQTT y CoAP para el mismo dato.

Nota técnica: CoAP usa UDP en lugar de TCP, lo que lo hace más liviano pero menos confiable. Es ideal para sensores con baterías o redes con ancho de banda limitado (NB-IoT, LoRa backhaul).

5.3 Integración del LLM local como controlador inteligente

Esta es la funcionalidad central de la Unidad 2. El LLM reemplaza las reglas *if/else* de Node-RED por razonamiento contextual en lenguaje natural.

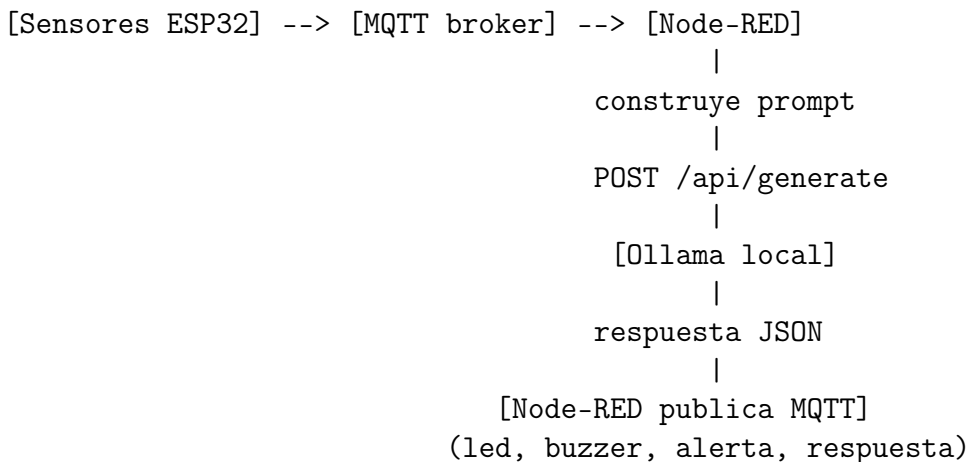
Herramienta recomendada: Ollama Ollama permite descargar y ejecutar modelos de lenguaje de forma local con una sola línea de comandos. Los modelos recomendados para esta práctica son **phi3:mini** (2,3 GB, muy rápido en CPU) o **llama3.2:3b** (2 GB).

```
# Instalación en Linux / macOS
curl -fsSL https://ollama.com/install.sh | sh

# Descargar modelo liviano
ollama pull phi3:mini

# Iniciar servidor (API en http://localhost:11434)
ollama serve
```

Arquitectura de integración:



Funcionamiento paso a paso:

1. Node-RED recibe los datos de los sensores cada 30 segundos o ante un evento.
2. Se construye un **prompt de contexto** con el estado actual del hogar. Ejemplo:

Eres el sistema de control de una casa inteligente.

Estado actual del hogar:

- Temperatura: 32 grados C (umbral normal: 28)
- Humedad: 65%
- Gas: 450 ppm (umbral de alerta: 400 ppm)
- Movimiento detectado: si
- Hora local: 02:15
- Persona detectada por camara: no

Analiza el estado y responde SOLO en formato JSON valido con exactamente esta estructura:

```
{
  "nivel_alerta": "normal|medio|critico",
  "activar_led": true|false,
  "activar_buzzer": true|false,
  "mensaje_alerta": "texto corto para la notificacion",
  "razonamiento": "explicacion breve de la decision"
}
```

3. Node-RED envía el prompt a la API de Ollama con un nodo HTTP Request:

POST http://localhost:11434/api/generate

```
{
  "model": "phi3:mini",
  "prompt": "<contexto generado dinamicamente>",
  "stream": false,
  "format": "json"
}
```

4. Node-RED parsea la respuesta y publica los comandos MQTT correspondientes:

```
smarthome/equipoXX/control/led      -> {"estado": true}
smarthome/equipoXX/control/buzzer   -> {"estado": true}
smarthome/equipoXX/llm/decision     -> {"nivel": "critico",
                                         "razonamiento": "..."}

```

Consejo de implementación: Usar "format": "json" en la petición a Ollama obliga al modelo a responder en JSON válido, lo que simplifica el parseo en Node-RED. Si el modelo no responde en JSON, revise que el prompt sea lo suficientemente explícito.

5.4 Interfaz de consulta en lenguaje natural

Agregar al *dashboard* de Node-RED un campo de texto donde el usuario pueda escribir preguntas en español sobre el estado del hogar. El sistema debe:

- Tomar la pregunta del usuario desde el *dashboard*.
- Inyectar automáticamente los datos actuales de los sensores como contexto.
- Enviar el prompt resultante al LLM local.
- Mostrar la respuesta en texto en el *dashboard*.

Ejemplos de preguntas que el sistema debe saber responder:

- “¿Es seguro dormir con estos niveles de gas?”
- “Resume el estado del hogar en una frase.”
- “¿Qué condición del hogar es más preocupante ahora mismo?”

5.5 Seguridad en redes IoT

Documentar en el informe técnico las siguientes medidas implementadas:

- Autenticación MQTT (usuario y contraseña) con `mosquitto_passwd`.
- Cifrado TLS en el transporte MQTT (puerto 8883).
- ACLs de Mosquitto que limiten qué clientes pueden publicar y suscribirse a qué tópicos.
- Análisis de al menos **una vulnerabilidad conocida** en el stack IoT del sistema (por ejemplo: MQTT sin autenticación, firmware sin actualización segura, broker expuesto en red pública) con su propuesta de mitigación.

6. Entregables

a) Prototipo funcional

Sistema completo en funcionamiento con MQTT seguro, CoAP operativo y LLM respondiendo consultas, demostrable en la sala durante la defensa.

b) Código fuente

- Firmware ESP32 actualizado con soporte TLS.
- Firmware ESP32 con cliente CoAP.
- Flujo de Node-RED actualizado exportado en formato `.json`.
- Archivo de configuración de Mosquitto (`mosquitto.conf`) y ACLs.
- Todo debe estar disponible en el mismo repositorio del proyecto, en una carpeta `unidad2/`.

c) Informe técnico (5–8 páginas adicionales a la Unidad 1)

El informe debe agregar las siguientes secciones a las ya entregadas:

1. Descripción del proceso de configuración de MQTT seguro (TLS + auth + ACLs).
2. Comparativa MQTT vs. CoAP: tamaño de mensaje, latencia, consumo de ancho de banda.
3. Arquitectura de integración del LLM: diagrama, prompt utilizado y ejemplos de respuestas.
4. Análisis de seguridad: vulnerabilidad identificada y medida de mitigación propuesta.
5. Capturas del *dashboard* con la interfaz de lenguaje natural funcionando.
6. Problemas encontrados en la integración del LLM y cómo se resolvieron.
7. Reflexión crítica: ¿en qué casos el LLM es más útil que una regla clásica, y en cuáles no?

d) Defensa oral (15 minutos)

- **8 minutos:** demostración en vivo, incluyendo una consulta en lenguaje natural al LLM.
- **7 minutos:** preguntas técnicas del profesor sobre protocolos y seguridad.
- Todos los integrantes deben participar activamente.

7. Rúbrica de evaluación - Unidad 2

La calificación final del proyecto se obtiene en escala de **1,0 a 7,0**. La nota mínima de aprobación es **4,0**. Cada criterio se evalúa en una escala interna de 1 a 4, ponderada según su peso relativo.

Criterio	Pond.	Logrado (4)	Parcial (2–3)	No logrado (1)
MQTT seguro (TLS + auth)	20 %	Broker activo en puerto 8883 con TLS y autenticación, ESP32 y Node-RED conectados correctamente	TLS configurado sin autenticación, o autenticación sin TLS	MQTT sin seguridad, igual que Unidad 1
ACLs y análisis de seguridad	10 %	ACLs definidas por cliente, vulnerabilidad analizada con mitigación documentada	ACLs presentes pero sin análisis de vulnerabilidad	Sin ACLs ni análisis
CoAP implementado	15 %	Sensor publica en CoAP, datos llegan a Node-RED, comparativa documentada en informe	CoAP configurado pero sin integración completa en Node-RED	No implementado
LLM local (Ollama)	25 %	LLM responde en JSON válido y los comandos MQTT se publican correctamente ante eventos reales	LLM responde pero los comandos MQTT no se publican o son incorrectos	LLM no integrado en el flujo
Consulta en lenguaje natural	15 %	Campo de texto en el <i>dashboard</i> funcional, respuestas coherentes con el estado actual	Interfaz presente pero respuestas fuera de contexto o inconsistentes	Sin interfaz de lenguaje natural
Informe técnico	10 %	Secciones nuevas completas, con diagrama de integración LLM y capturas	Faltan secciones o capturas relevantes	Informe no actualizado
Defensa oral	5 %	Todos participan, demuestran comprensión del LLM y de los protocolos	Participación desigual o explicaciones superficiales	Sin defensa o sin dominio técnico
Total	100 %			

Cuadro 1: Rúbrica de evaluación del proyecto — Unidad 2

Conversión de puntaje a nota

$$\text{Nota} = 1,0 + \frac{\text{Puntaje obtenido}}{100} \times 6,0$$

Puntaje obtenido (%)	Nota
100	7,0
83	6,0
67	5,0
50	4,0 (mínimo para aprobar)
33	3,0
17	2,0
0	1,0

Cuadro 2: Tabla de conversión puntaje-nota

Unidad 3: IoT en la Industria - Gemelo Digital y Agente Autónomo

1. Introducción

En esta fase final el sistema SmartHome evoluciona hacia una arquitectura de nivel productivo. Se incorporan tres capacidades clave: un **gemelo digital** del hogar que mantiene una representación virtual sincronizada con los sensores físicos; un **agente IoT autónomo** basado en el LLM local que no solo responde preguntas sino que ejecuta acciones en cadena de forma proactiva; y un **stack de servicios en contenedores** que replica la arquitectura de plataformas industriales como AWS IoT Core o Azure IoT Hub, pero corriendo completamente en la red local de sus dispositivos.

Estas tres características están directamente relacionadas con aplicaciones industriales reales: monitoreo de plantas de manufactura, gestión de edificios inteligentes, microrredes eléctricas y agricultura de precisión, entre otras.

2. Problema a resolver

La familia y su equipo quieren llevar el hogar inteligente al siguiente nivel:

- **Predicción:** El sistema debe poder anticipar cuándo superará umbrales críticos basándose en tendencias históricas, antes de que ocurra el evento.
- **Agente autónomo:** El LLM debe poder ejecutar secuencias de acciones por sí mismo (activar actuadores, enviar notificaciones, ajustar umbrales) sin intervención manual.
- **Gemelo digital:** Una representación virtual del hogar actualizada en tiempo real que sirve como contexto enriquecido para el agente.
- **Infraestructura containerizada:** Todo el stack (broker, base de datos, visualización, orquestación) debe desplegarse con Docker Compose en una sola máquina.

3. Objetivo

Implementar un sistema IoT de nivel industrial que integre un gemelo digital, predicción de series de tiempo y un agente autónomo basado en LLM local, desplegado completamente en contenedores Docker sobre la red del laboratorio.

4. Objetivos específicos

1. Desplegar el stack completo de servicios (Mosquitto, InfluxDB, Grafana, n8n, Node-RED) usando Docker Compose.
2. Implementar un gemelo digital del hogar como objeto JSON persistente sincronizado con los sensores en tiempo real.
3. Exponer el estado del gemelo digital a través de una API REST accesible por el agente LLM.
4. Desarrollar un script de predicción de series de tiempo para temperatura y gas con proyección a 30 minutos.
5. Implementar el agente autónomo en n8n, capaz de ejecutar acciones en cadena a partir del razonamiento del LLM.

6. Configurar un *dashboard* en Grafana con visualización histórica, predicciones y log de decisiones del agente.
7. Analizar cómo la arquitectura del sistema puede escalarse o adaptarse a un contexto industrial real.
8. Documentar el sistema completo (Unidades 1, 2 y 3) en un informe técnico consolidado.

5. Funcionalidades mínimas requeridas

5.1 Stack de servicios con Docker Compose

Todo el sistema debe poder iniciarse con un único comando `docker compose up -d`. El archivo `docker-compose.yml` debe incluir los siguientes servicios:

```
version: '3.8'
services:
  mosquitto:
    image: eclipse-mosquitto
    ports: ["8883:8883"]
    volumes: ["/mosquitto:/mosquitto/config"]

  influxdb:
    image: influxdb:2.7
    ports: ["8086:8086"]
    environment:
      DOCKER_INFLUXDB_INIT_MODE: setup
      DOCKER_INFLUXDB_INIT_ORG: smarthome
      DOCKER_INFLUXDB_INIT_BUCKET: sensores

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
    depends_on: [influxdb]

  n8n:
    image: n8nio/n8n
    ports: ["5678:5678"]

  nodered:
    image: nodered/node-red
    ports: ["1880:1880"]
```

Nota técnica: Ollama no necesita correr dentro de Docker; puede ejecutarse directamente en el sistema operativo del host y ser accedido desde los contenedores usando la dirección `host.docker.internal:11434` (en Linux: `172.17.0.1:11434`).

5.2 Gemelo Digital del Hogar

El gemelo digital es un objeto JSON persistente que mantiene el estado completo y el historial reciente del hogar. Se implementa como un nodo de almacenamiento en Node-RED o como un archivo JSON actualizado periódicamente. Debe:

- Mantenerse sincronizado con los datos MQTT en tiempo real.
- Almacenar los últimos 60 registros de cada sensor (historial de 1 hora a resolución de 1 minuto).
- Exponer el estado completo a través de una **API REST** en Node-RED (endpoint `GET /gemelo/estado`).
- Incluir un campo `resumen_llm` que el agente actualice con cada ciclo de razonamiento.

Ejemplo de estructura del gemelo digital:

```
{
  "ultimo_update": "2025-06-10T22:15:00",
  "estado_actual": {
    "temperatura": 29.5,
    "humedad": 70,
    "gas": 420,
    "movimiento": true,
    "persona": false,
    "led": false,
    "buzzer": false
  },
  "historial_1h": [
    {"ts": "2025-06-10T21:15:00", "temperatura": 27.0,
    "gas": 310},
    {"ts": "2025-06-10T21:30:00", "temperatura": 28.1,
    "gas": 350}
  ],
  "alertas_activas": ["gas_alto"],
  "prediccion_30min": {
    "temperatura": 31.2,
    "gas": 480
  },
  "resumen_llm": "Temperatura en aumento. Gas sobre umbral."
}
```

5.3 Predicción con series de tiempo

Implementar un script Python que se ejecute cada 10 minutos y realice lo siguiente:

1. Consultar el historial de las últimas 6 horas desde InfluxDB (o desde el archivo CSV de la Unidad 1).
2. Calcular una proyección a 30 minutos usando **regresión lineal** (`numpy.polyfit`) o la librería **Prophet** para temperatura y gas.
3. Publicar la predicción en el broker MQTT:

```
smarthome/equipoXX/prediccion/temperatura -> {"valor": 31.2,
                                              "horizon_min": 30}
smarthome/equipoXX/prediccion/gas         -> {"valor": 480,
                                              "horizon_min": 30}
```

4. Si la predicción supera el umbral antes de 30 minutos, publicar adicionalmente una **alerta preventiva** en `smarthome/equipoXX/alerta` con el campo `"tipo": "preventiva"`.
5. Mostrar en Grafana un panel con los valores reales y la proyección superpuesta.

Alternativa liviana: Si el equipo no quiere instalar Prophet, es suficiente con `numpy.polyfit` de grado 1 (regresión lineal simple) sobre los últimos 20 valores. El objetivo es demostrar el concepto de predicción, no la precisión del modelo.

5.4 Agente IoT Autónomo con n8n y Ollama

El agente es un flujo en **n8n** que se activa cada 5 minutos o ante un evento MQTT crítico. A diferencia del controlador de la Unidad 2, el agente puede ejecutar **acciones en cadena** de forma autónoma.

Flujo del agente:

1. **Trigger:** Timer de 5 minutos o Webhook desde Node-RED ante un evento crítico.
2. **Obtener contexto:** n8n consulta la API REST del gemelo digital (estado actual + historial + predicción).
3. **Razonar:** n8n envía el contexto al LLM con un prompt de agente. El LLM devuelve una lista de acciones a ejecutar:

Eres un agente de control de un hogar inteligente.

Tienes disponibles las siguientes herramientas:

- `activar_actuador(dispositivo, estado)`
- `enviar_notificacion(mensaje)`
- `registrar_evento(descripcion)`
- `ajustar_umbral(sensor, nuevo_umbral)`

Estado del gemelo digital (JSON): `<contexto>`

Decide que acciones ejecutar. Responde SOLO en JSON:

```
{
  "acciones": [
    {
      "herramienta": "activar_actuador",
      "parametros": {"dispositivo": "led", "estado": true}
    },
    {
      "herramienta": "enviar_notificacion",
      "parametros": {"mensaje": "Gas critico, verificar cocina"}
    }
  ],
  "razonamiento": "El gas supera 400 ppm con tendencia..."
}
```

4. **Ejecutar:** n8n parsea el array de acciones y ejecuta cada una: publica en MQTT, llama a la API de Telegram, escribe en InfluxDB o en el CSV.
5. **Registrar:** El razonamiento del agente y las acciones ejecutadas se guardan en InfluxDB y se muestran en Grafana.

Nota técnica: n8n tiene un nodo nativo *HTTP Request* que permite llamar a la API de Ollama directamente. Para parsear el JSON de respuesta se puede usar el nodo *Code* con JavaScript. No se requiere instalar ningún plugin adicional.

5.5 Dashboard en Grafana

Reemplazar o complementar el *dashboard* de Node-RED con **Grafana** conectado a InfluxDB:

- Panel de series de tiempo para temperatura, humedad y gas (últimas 6 horas).
- Panel de predicción: valores reales más proyección a 30 minutos superpuesta.
- Panel de log de decisiones del agente (tabla con *timestamp*, razonamiento y acciones).
- Panel de estado del gemelo digital (tabla con el último valor de cada variable).
- Al menos **una alerta de Grafana** configurada que dispare una notificación (correo o webhook) cuando temperatura o gas supere el umbral.

5.6 Análisis de aplicación industrial

Como parte de la Unidad 3, cada equipo debe investigar y documentar cómo su sistema SmartHome podría escalarse a un entorno industrial. Elegir uno de los siguientes contextos:

- **Monitoreo de sala de servidores:** temperatura, humedad, acceso físico, UPS.
- **Automatización agrícola:** sensores de suelo, riego automático, invernadero.
- **Microrred eléctrica:** monitoreo de consumo, generación solar, baterías.
- **Edificio inteligente:** control de acceso, climatización (HVAC), eficiencia energética.

La sección en el informe debe incluir: diagrama de arquitectura adaptada al contexto industrial, protocolos recomendados a esa escala, principales desafíos de escalabilidad, y cómo el agente LLM aportaría valor operacional en ese entorno.

6. Entregables

a) Prototipo funcional

Sistema completo en funcionamiento: Docker Compose levantado, gemelo digital sincronizado, predicción publicada en MQTT y agente tomando decisiones demostrables en sala.

b) Código fuente

- Archivo `docker-compose.yml` funcional con todos los servicios.
- Script Python de predicción de series de tiempo.
- Flujo n8n del agente autónomo exportado en formato `.json`.
- Dashboard Grafana exportado en formato `.json`.
- Flujo Node-RED actualizado (gemelo digital + API REST) en formato `.json`.
- Todo debe estar en el repositorio del proyecto, en una carpeta `unidad3/`.

c) Informe técnico final consolidado (20–30 páginas, incluye Unidades 1, 2 y 3)

El informe final debe contener todas las secciones de la Unidad 1 y 2, más:

1. Arquitectura del stack Docker Compose: diagrama de servicios y redes.
2. Diseño e implementación del gemelo digital.
3. Descripción del script de predicción: método utilizado, gráficos de predicción vs. valor real.
4. Diseño del agente autónomo: prompt de agente, herramientas disponibles, ejemplos reales de decisiones tomadas durante las pruebas.
5. Descripción del dashboard Grafana con capturas de pantalla.
6. Análisis del contexto industrial elegido con diagrama de arquitectura adaptada.
7. Reflexión crítica: limitaciones del LLM como controlador IoT (latencia, no-determinismo, fallos en el parseo de JSON) y cuándo es preferible una regla clásica.
8. Conclusiones generales del proyecto integrador.

d) Defensa oral final (20 minutos)

- **10 minutos:** demostración en vivo del sistema completo, incluyendo al agente tomando al menos una decisión autónoma visible.
- **10 minutos:** preguntas técnicas sobre cualquier aspecto de las tres unidades.
- El equipo debe poder explicar el razonamiento del agente en una decisión específica ocurrida durante las pruebas.
- Todos los integrantes deben participar activamente.

7. Rúbrica de evaluación - Unidad 3

La calificación final del proyecto se obtiene en escala de **1,0 a 7,0**. La nota mínima de aprobación es **4,0**. Cada criterio se evalúa en una escala interna de 1 a 4, ponderada según su peso relativo.

Criterio	Pond.	Logrado (4)	Parcial (2–3)	No logrado (1)
Stack Docker Compose	15 %	Todos los servicios levantados y comunicados entre sí correctamente	Algunos servicios sin integrar o con errores de red	Docker no implementado
Gemelo digital	20 %	Sincronizado en tiempo real, API REST funcional, historial disponible	Gemelo estático o sin API REST expuesta	No implementado
Predicción de series de tiempo	15 %	Predicción publicada en MQTT y visualizada en Grafana con comparación vs. valor real	Predicción calculada pero no integrada al sistema	Sin predicción
Agente autónomo (n8n + LLM)	25 %	Agente ejecuta correctamente acciones en cadena ante eventos reales demostrables	Agente razona correctamente pero no ejecuta todas las acciones	Agente no implementado
Dashboard Grafana	10 %	Completo con historial, predicción, log de agente y alerta configurada	Dashboard sin predicción o sin alerta configurada	Sin Grafana
Análisis industrial	10 %	Diagrama de arquitectura adaptada, protocolos justificados, análisis de escalabilidad	Descripción del contexto sin diagrama ni análisis de protocolos	Sección ausente
Informe y defensa final	5 %	Informe consolidado completo, demostración del agente fluida y explicada	Faltan secciones del informe o la demostración es incompleta	Sin entrega o sin defensa
Total	100 %			

Cuadro 3: Rúbrica de evaluación del proyecto — Unidad 3

Conversión de puntaje a nota

El puntaje ponderado total se calcula sobre 100 puntos y se convierte a nota según la siguiente escala lineal, con nota mínima 1,0 y máxima 7,0:

$$\text{Nota} = 1,0 + \frac{\text{Puntaje obtenido}}{100} \times 6,0$$

Puntaje obtenido (%)	Nota
100	7,0
83	6,0
67	5,0
50	4,0 (mínimo para aprobar)
33	3,0
17	2,0
0	1,0

Cuadro 4: Tabla de conversión puntaje-nota